Questions probables du jury — réponses courtes

Épreuve CDA (REV-CDA-V04) : 40 min de présentation, puis **45 min d'entretien technique** sur la base du dossier et de la présentation, avec un questionnement complémentaire sur les compétences que le projet ne couvre pas. Puis 30 min de questionnaire professionnel (documentation technique en anglais) et 20 min d'entretien final.

Chaque réponse tient en deux ou trois phrases. Le § renvoie au dossier, pour y retourner si le jury veut le détail.

Trois règles pour l'oral

1. Réponds d'abord à la question posée, puis justifie. Pas l'inverse.
2. Si tu ne sais pas : « Je ne l'ai pas traité dans ce projet. Ce que je ferais, c'est… ». Le jury sanctionne l'invention, pas la lacune.
3. Quand tu as fait une erreur et que tu l'as corrigée, raconte-la. C'est ce qui prouve la compétence mieux qu'une réussite.

---

1. Architecture et choix techniques

Pourquoi Vue 3 alors que le cahier des charges recommande React ? Le cahier des charges recommande, il n'impose pas. Vue 3 avec la Composition API offre le même modèle de composants, ma maîtrise plus avancée réduisait le risque sur un projet mené seule, et l'intégration de CodeMirror 6 y est directe. Le découpage vues / composants / services / store reste transposable à React. (§ 5.2)

Pourquoi Ollama en local plutôt que l'API OpenAI ? Le cahier des charges autorise explicitement les deux. L'exigence RGPD — aucune donnée personnelle transmise sans anonymisation — est intenable sur des textes rédigés librement, où l'anonymisation n'est jamais exhaustive. En hébergeant qwen2.5:3b localement, les contenus ne quittent pas l'infrastructure : l'exigence est satisfaite par construction. (§ 5.2)

Quels sont les inconvénients de ce choix ? Latence plus élevée, qualité inférieure à GPT-4, un prérequis d'installation, et un service de plus à maintenir — la CVE-2026-7482 sur Ollama l'a confirmé. Héberger soi-même, c'est aussi maintenir soi-même. La couche `ia_service` isole l'appel, donc basculer vers un fournisseur distant ne toucherait pas les routes. (§ 5.2, § 11)

Pourquoi trois couches ? Quelle est la règle qui les sépare ? Routes → services → persistance. La règle : **aucun import Flask dans `services/`**. C'est elle qui rend les

composants métier testables sans démarrer de serveur ni ouvrir de base, donc elle conditionne tout le plan de tests unitaires. (§ 5.2, § 9)

Cette règle est-elle respectée partout ? Presque. La lecture de la configuration passe par `current_app`, qui est un objet Flask. C'est le seul point où elle fléchit, et c'est écrit dans le dossier. (§ 7.2)

Pourquoi Markdown plutôt qu'un WYSIWYG ? Le cahier des charges laisse le choix. Le Markdown est stocké tel quel, sans HTML intermédiaire : la surface d'attaque XSS est réduite, les échanges avec le modèle portent sur du texte brut, et les différences avant/après restent lisibles dans l'historique IA. (§ 5.2)

Pourquoi le navigateur n'appelle-t-il pas Ollama directement ? Parce que ni le quota ni la traçabilité RGPD ne seraient applicables : l'utilisateur contournerait le quota en appelant le port 11434 lui-même. L'API contrôle l'authentification, la propriété du document et le quota, puis journalise. (§ 5.2)

Comment feriez-vous évoluer l'application vers plusieurs utilisateurs sur un même document ? Ce n'est pas dans le périmètre, et c'est volontaire. Techniquement il faudrait une table de partage, un contrôle d'accès par rôle sur le document, et une résolution de conflits (CRDT ou verrou optimiste). Et surtout : la faille XSS de l'aperçu deviendrait une prise de contrôle de compte dès le premier document partagé. (§ 2, § 7.1)

---

2. Base de données

Pourquoi des UUID plutôt que des entiers auto-incrémentés ? Les identifiants ne sont pas énumérables : on ne balaie pas `/documents/1`, `/documents/2`. C'est de la défense en profondeur, en plus du filtrage par propriétaire. Contrepartie assumée : 36 octets par clé et des index moins compacts qu'un entier. (§ 5.4, § 8)

Pourquoi déclarer les cascades au niveau du SGBD et pas seulement dans l'ORM ? Pour que l'intégrité tienne même si quelqu'un écrit avec un client SQL. Une contrainte qui n'existe que dans le code applicatif n'est pas une contrainte. C'est aussi ce qui garantit le droit à l'effacement RGPD : supprimer un compte supprime documents, dossiers, interactions IA, consentements et sessions, sans code dédié. (§ 5.4, § 8)

Combien de tables, et à quoi sert chacune ? Sept dans le modèle du dossier : `user`, `dossier`, `document`, `ia` (journal des interactions), `consentement`, `user_session` (refresh tokens) et `password_reset`. Le script en crée une huitième, `journal_admin`, qui trace les actions d'administration — elle a été ajoutée après la rédaction du § 5.4 et n'y figure pas. Si la question vient, dis-le comme ça : c'est une omission de documentation, pas une incohérence

de code. (§ 5.4, § 5.5)

Pourquoi SQLite en développement et MySQL en cible ? SQLAlchemy 2 permet de travailler sur SQLite en local et de viser MySQL sans réécrire la couche d'accès. Le script `schema_mysql.sql` a été exécuté et vérifié sur un serveur réel ; la bascule complète attend une machine avec un démon Docker. (§ 5.5, § 4)

Quelles différences avez-vous dû gérer entre SQLite et MySQL ? Les types (`CHAR(36)` pour les UUID), les contraintes `CHECK` que SQLite ignore longtemps, et l'authentification `caching_sha2_password` de MySQL 8.4, qui impose le paquet `cryptography` à côté de PyMySQL — sans lui, la première connexion de la pile échoue. (§ 6)

Avez-vous mis des index ? Lesquels et pourquoi ? Oui, et des index **composites**, calqués sur les requêtes chaudes : `(user_id, updated_at)` sur `document` pour la liste paginée triée, `(user_id, created_at)` sur `ia` pour le comptage du quota sur 24 h, `(id_document, created_at)` pour l'historique d'un document. Un index sur la seule clé étrangère n'aurait pas servi le tri. (§ 5.4, § 5.5)

NoSQL : pourquoi ne pas l'avoir utilisé ? Les données sont fortement relationnelles — un utilisateur, ses dossiers, ses documents, son journal — avec des contraintes d'intégrité que je voulais porter par le schéma. Un document store aurait eu du sens pour le seul journal `ia` si le volume l'avait exigé.

---

3. Sécurité

Expliquez votre mécanisme d'authentification. Mot de passe haché en bcrypt salé. À la connexion, deux jetons : un access token JWT HS256 de 15 minutes transmis en en-tête `Authorization`, jamais stocké en base ; et un refresh token de 64 octets aléatoires dans un cookie `httpOnly / Secure / SameSite=Strict / path=/auth`, valable 7 jours, dont seule l'empreinte SHA-256 est stockée. (§ 8)

Pourquoi deux jetons ? Deux besoins contradictoires. Un jeton court est sûr mais oblige à se reconnecter sans arrêt ; un jeton long est confortable mais dangereux. La combinaison donne les deux : 15 minutes d'exposition maximale si l'access token fuite, et une révocation immédiate possible côté serveur sur le refresh token. (§ 8)

Un JWT est censé être sans état. Pourquoi une table de sessions ? Parce qu'un JWT seul ne se révoque pas avant son expiration. La table `user_session` rend la révocation possible. J'assume le compromis : je perds le « sans état » sur le refresh, je le garde sur l'access token, qui est celui qui circule à chaque requête. (§ 4, § 8)

Comment détectez-vous un vol de jeton ? Par la rotation. Chaque usage marque le jeton consommé et en émet un nouveau. Si un jeton déjà consommé se représente, soit l'utilisateur rejoue une vieille requête, soit il y a vol : dans le doute, je révoque toutes les sessions du compte. (§ 7.2, § 8)

Pourquoi la purge filtre-t-elle sur l'expiration et non sur le drapeau `revoke` ? Parce qu'une session révoquée doit survivre jusqu'au terme de son jeton. Si je la supprimais, un jeton volé puis rejoué ne serait plus reconnu comme un rejeu mais comme un jeton inconnu — et la détection de vol tomberait. (§ 7.2)

Comment empêchez-vous un utilisateur de lire les documents d'un autre ? La propriété est dans la clause `WHERE` : je demande à la base un document qui a cet identifiant **et** qui appartient à l'utilisateur courant. Je ne charge pas puis ne compare pas. Oublier la condition fait échouer un test ; oublier un `if` ouvrirait une faille. (§ 7.3)

Pourquoi un 404 et pas un 403 ? Un 403 confirmerait que la ressource existe. Le 404 est indiscernable d'un identifiant inexistant. (§ 5.7, § 8)

Comment protégez-vous le back-office ? Le rôle n'est pas lu dans le JWT, il est relu en base à chaque appel `/admin`. Un jeton émis avant une rétrogradation resterait sinon valide jusqu'à son expiration. La garde du routeur côté client est du confort d'usage ; le contrôle qui compte est serveur. (§ 5.2, § 7.4)

Parlez-nous de l'injection SQL. Aucune requête n'est construite par concaténation ni par f-string : tout passe par `select()` et des comparaisons sur attributs de modèle, que SQLAlchemy transmet en paramètres liés. C'est vérifiable par lecture — pas un seul `text(` dans le projet — et vérifié à l'exécution par le cas TS-02, qui soumet `' OR 1=1 --` dans les champs de recherche, de titre et d'email. (§ 8)

Et le XSS ? C'est la vraie faille que j'ai trouvée et corrigée. L'aperçu Markdown injectait le résultat de `marked` par `v-html` sans filtrage : un `<img src=x onerror=...>` saisi dans le texte était exécuté. Et l'enregistrement automatique la rendait **stockée**, donc rejouée à chaque ouverture. Correction : assainissement de la sortie par DOMPurify, profil `html`. (§ 7.1)

Pourquoi assainir la sortie et pas l'entrée ? Parce que le Markdown reste stocké tel quel : un changement de politique d'affichage ne demande pas de réécrire les documents existants. Et trois lignes de défense se superposent — stockage en Markdown brut, assainissement DOMPurify, cookie `httpOnly` — ce qui compte, c'est qu'elles soient indépendantes. (§ 7.1, § 8)

Et le CSRF ? Les routes métier s'autorisent par l'en-tête `Authorization`, que le navigateur n'ajoute jamais automatiquement à une requête inter-site : elles ne sont pas exposées. Le seul point exposé est `/auth/refresh`, qui s'authentifie par cookie — couvert par `SameSite=Strict`, `path=/auth` et une liste CORS explicite, jamais `*`. (§ 8)

Pourquoi pas de jeton CSRF en double-submit ? Je l'ai étudié et écarté : il n'aurait rien ajouté ici, puisqu'un XSS réussi permettrait de lire le cookie et de forger l'en-tête. Cette protection suppose le XSS déjà fermé — les deux sujets sont liés. (§ 8)

Comment gérez-vous la limitation de débit ? Flask-Limiter, avec trois seuils différents parce que les menaces diffèrent : 5/min sur `/login` contre le bourrage d'identifiants, et une limite horaire sur `/register` et `/forgot-password` contre l'abus de ressource. Le 429 est normalisé en JSON et volontairement peu bavard. (§ 6, § 8)

Empêchez-vous l'énumération de comptes ? Oui. Sur `/forgot-password`, la réponse est identique que l'adresse existe ou non. Sur `/login`, un mot de passe erroné et un compte inexistant renvoient tous deux 401 avec le même message. (§ 8)

Où sont vos secrets ? Nulle part dans le code. `.env` est exclu par `.gitignore`, `.env.example` documente toutes les variables attendues, et l'application **refuse de démarrer** sans `JWT_SECRET_KEY` — pas de valeur de repli, parce qu'une clé de secours en dur est une clé en production. (§ 6, § 8)

Quelle est votre plus grosse faiblesse de sécurité aujourd'hui ? A09 de l'OWASP : la journalisation applicative est incomplète. Le journal des interactions IA existe, mais il n'y a pas de journalisation systématique des événements de sécurité — connexions échouées, révocations, changements de rôle. C'est écrit dans le dossier. (§ 8)

---

4. Qualité, tests, jeu d'essai

Décrivez votre stratégie de test. Quatre niveaux, du moins coûteux au plus coûteux : analyse statique, tests unitaires, tests d'intégration — les trois à chaque push — puis tests fonctionnels manuels en fin d'itération. Trois zones concentrent l'effort : la chaîne d'authentification, le contrôle de propriété et le quota. (§ 9)

Vos chiffres ? 195 cas automatisés tous au vert en 59 secondes, campagne du 3 octobre 2026 : 146 backend (50 unitaires, 79 d'intégration, 17 de sécurité) et 42 Vitest, plus 7 ajoutés sur les correctifs. Couverture backend 95 %, dont 100 % sur `services/` et la persistance. (§ 9)

Pourquoi seulement 62 % côté frontend ? Le chiffre est tiré vers le bas par cinq modules de service qui sont des enveloppes d'une ligne autour d'axios : les tester reviendrait à tester axios. Mon objectif était 80 % sur `services/` et sur les routes, sans objectif global — viser un pourcentage global pousse à écrire des tests faciles sur du code sans risque. (§ 9)

Comment garantissez-vous l'indépendance des tests ? `create_all` puis `drop_all` autour de chaque cas : une base recréée à chaque test. L'ordre d'exécution n'a aucune incidence, et un

test qui échoue ne fait pas échouer le suivant. Des tests qui ne passent que dans un certain ordre ne prouvent rien. (§ 9)

Comment testez-vous sans appeler Ollama ? `call_ollama` est remplacé par un double. Mais j'ai appris que **les doubles mentent** : trois défauts de la chaîne IA sont passés sous un faux `requests.post` qui acceptait `json()` à tout moment et ne fermait jamais de connexion. Il a fallu un vrai `http.server` imitant `/api/generate` pour les voir. (§ 9)

Comment savez-vous que vos tests sont bons ? Par mutation : je casse volontairement le code pour vérifier que le test échoue. Un test qu'on n'a jamais vu échouer ne prouve rien. (§ 9)

Pourquoi avoir choisi la génération IA pour le jeu d'essai ? Elle traverse les trois couches et sollicite un système externe, elle concentre cinq contrôles de nature différente — authentification, validation, autorisation, quota, disponibilité — et elle porte les deux exigences les plus délicates du cahier des charges. C'est aussi la seule qu'aucune capture d'écran ne peut attester. (§ 10)

Qu'est-ce qui fait la qualité de votre jeu de données ? Une ligne. Le journal du compte U1 contient 19 interactions de moins de 24 h **et une datée de 25 heures**. C'est le seul moyen de distinguer une fenêtre glissante d'un compteur journalier : sans elle, le cas JE-07 passerait quelle que soit l'implémentation. Construire un jeu de données, c'est choisir les valeurs qui font échouer les mauvaises implémentations. (§ 10)

Qu'est-ce qui n'est pas testé ? Les tests système TS-01 à TS-10 attendent la pile conteneurisée sur une machine avec un démon Docker ; les tests d'acceptation attendent le point de validation ; les tests de charge viennent après. Ce sont mes trois seuls critères de sortie non satisfaits. (§ 9)

---

5. Gestion de projet et DevOps

Quelle méthode avez-vous suivie ? Des itérations d'une semaine, chacune close par un lot utilisable de bout en bout, et un découpage en **lots verticaux** — une fonctionnalité qui traverse le front, l'API et la base — plutôt qu'en couches horizontales. Avec des couches, on ne valide rien avant l'assemblage final. (§ 4)

Comment travailliez-vous, seule ? J'ai remplacé les rituels d'équipe par un point hebdomadaire avec mon référent pédagogique, qui tenait le rôle de maîtrise d'ouvrage, et des revues de code croisées avec la promotion sur les parties sensibles — en premier lieu la chaîne d'authentification. (§ 3, § 4)

Comment gérez-vous les versions ? Une branche par fonctionnalité, fusionnée dans `main` après relecture, messages de commit conventionnés (`feat:`, `fix:`, `docs:`, `refactor:`). `.env`, les environnements virtuels, `node_modules` et la base SQLite sont exclus par `.gitignore`. (§ 4)

Pourquoi commiter `package-lock.json` ? Ce n'est pas une question d'organisation mais une mesure de sécurité : c'est le seul moyen de savoir, après coup et avec certitude, quelle version exacte d'une dépendance a été installée. La compromission d'axios de mars 2026 l'a démontré. (§ 4, § 11)

Décrivez votre pipeline d'intégration continue. GitHub Actions sur chaque push et chaque pull request, trois travaux **parallèles** : backend (Ruff analyse et format, pytest avec des clés éphémères), frontend (ESLint, Prettier, vue-tsc, Vitest, build), et Docker (validation de la composition, construction des deux images). (§ 6)

Pourquoi des travaux parallèles ? Ce n'est pas une optimisation de performance : enchaînés, un échec du frontend masquait l'état du backend, et il fallait deux exécutions pour connaître les deux. (§ 6)

Qu'est-ce que `npm ci --ignore-scripts` apporte ? `npm ci` installe exactement ce que décrit le fichier de verrouillage. `--ignore-scripts` empêche les scripts de cycle de vie npm d'exécuter du code arbitraire à l'installation : c'est la leçon directe de l'incident axios. (§ 6, § 11)

Comment est conteneurisée l'application ? Trois services dans `docker-compose.yml` : MySQL 8.4 avec volume persistant et sonde de santé ; le backend sous Gunicorn avec un utilisateur non privilégié ; le frontend construit par Vite puis servi par Nginx, image finale d'environ 50 Mo. Les secrets sont déclarés `${VAR:?message}` : la composition échoue si la variable manque, au lieu de démarrer avec une valeur inventée. (§ 6)

Pourquoi Ollama n'est-il pas dans la composition ? Son image pèse plusieurs gigaoctets et le modèle se télécharge à part : le garder hors du cycle de construction évite de le re-télécharger à chaque reconstruction d'image. Il est atteint par `host.docker.internal`. (§ 6)

Qu'est-ce qui reste à faire avant une mise en production ? Une liste de contrôle, et ce sont des changements de configuration, pas de code — c'est précisément pourquoi ils s'oublient : `APP_ENV=production` pour activer `Secure` sur le cookie, une `JWT_SECRET_KEY` distincte, `CORS_ORIGINS` avec le domaine réel, un compte MySQL aux droits limités sans DDL, un vrai SMTP, la terminaison TLS, et Redis pour Flask-Limiter. (§ 6)

Pourquoi Redis pour Flask-Limiter ? Parce que le stockage en mémoire compte par worker, et Gunicorn en lance plusieurs : avec deux workers, le seuil de `/auth/login` est doublé. (§ 6)

Un piège de déploiement que vous avez anticipé ? `SameSite=Strict` raisonne en sites

(eTLD+1), pas en origines. En développement, front et API sont tous deux sur localhost, donc le cookie passe. En production il faut deux sous-domaines d'un même domaine, ou l'API derrière un reverse proxy — deux domaines distincts casseraient le rafraîchissement. (§ 6)

---

6. Accessibilité, RGPD, éco-conception, veille

Qu'avez-vous fait pour l'accessibilité ? Audit axe-core le 3 octobre sur 15 écrans, référentiel RGAA 4.1.2. 120 occurrences au départ, dont 117 dues à une seule couleur : l'accent `#E0533C` ne tenait pas le 4,5:1. L'assombrir en `#C4341C` a corrigé 64 violations. Bilan après correction : 0 violation, WCAG 2.0 et 2.1 niveaux A et AA. (§ 9)

« 0 violation », c'est donc conforme ? Non, et c'est la nuance qui compte. axe-core mesure des contrastes et repère des attributs manquants, il ne sait pas dire si un libellé est compréhensible ni si l'ordre de tabulation suit l'ordre de lecture. Mes contrôles manuels ont relevé six points que l'outil n'a pas vus — titre de page identique sur 16 écrans, pas de lien d'évitement, `:focus-visible` global manquant. Chiffrés à trois heures et demie, à traiter. (§ 9)

Comment traitez-vous le RGPD ? Minimisation — quatre données d'identité, aucune facturation ni géolocalisation. Consentement prouvable, tracé en base avec type, réponse et horodatage ; l'inscription est refusée sans lui. Droit à l'effacement garanti par les cascades du schéma, pas par du code. Aucun transfert : l'inférence est locale. Pages légales intégrées, et un export des données. (§ 8)

Une réserve sur le RGPD ? Oui, et elle est dans le dossier : le journal `ia` conserve `content_before` et `content_after`, donc des extraits des documents. C'est nécessaire à la traçabilité et au marquage AI Act, mais ça double le stockage des contenus. Une durée de conservation explicite reste à définir. (§ 8)

L'AI Act vous concerne-t-il ? Oui, depuis le 2 août 2026, sur deux obligations de transparence : informer que l'utilisateur interagit avec une IA, et marquer les contenus générés. Partiellement couvert — le panneau est intitulé « Assistant IA — Ollama » et rien n'est appliqué sans décision de l'utilisateur. Trois compléments restent à produire. (§ 11)

Qu'est-ce que votre veille a changé concrètement ? Huit entrées sur six mois, trois actions. DOMPurify mis à jour en 3.4.16 — la version du projet était affectée. Ollama mis à jour en 0.35.1 et la règle d'écoute sur localhost passée du statut d'habitude implicite à celui de consigne écrite. Et les compléments AI Act identifiés. (§ 11)

Quelle est votre leçon de veille ? La sévérité publiée ne prédit pas l'impact réel : la CVE la plus sévère de ma liste (9,1) était atténuée par ma configuration, la plus modérée (5,1) affectait directement ma version. Et une mesure de sécurité est un composant comme un

autre : ajouter DOMPurify n'a pas clos le sujet XSS, ça l'a déplacé vers la maintenance de DOMPurify. (§ 11)

Et l'éco-conception ? Chargement différé par route, donc un fragment par écran. Dépendances contenues : neuf paquets backend. Et une mesure plutôt qu'une impression — daisyUI retiré, 45,6 Ko pour cinq classes réellement utilisées : le CSS passe de 79,9 à 34,4 Ko bruts, 57 % de moins. Le menu mobile réécrit y a gagné `aria-expanded` et la fermeture par Échap. (§ 6)

---

7. Questions pièges — et la bonne posture

« Vous n'avez pas déployé, donc le projet n'est pas fini ? » La conteneurisation et le pipeline sont en place depuis le 4 octobre et le schéma MySQL a été exécuté et vérifié sur un serveur réel. Ce qui manque est une machine disposant d'un démon Docker, pas une étape de conception. La liste de contrôle de mise en production est écrite. (§ 4, § 6)

« Pourquoi avoir gardé une erreur dans le dossier ? » Parce qu'elle est instructive. Mes textes d'interface avaient dérivé vers un positionnement « juridique » absent du cahier des charges, et le dernier résidu n'était pas visible : il était dans les gabarits de prompt du backend. Invisible, mais envoyé au modèle à chaque appel, donc avec un effet réel sur les sorties. Un prompt est du code, il se teste comme du code — c'est le cas TU-04. (§ 4, § 9)

« Le quota, ça se contourne en créant plusieurs comptes. » Oui. Le quota du cahier des charges est par utilisateur, pas par personne. Les contre-mesures seraient la vérification de l'adresse à l'inscription et une limite par adresse IP ou par domaine — la limitation de débit sur `/register` en couvre déjà l'abus massif. (§ 8)

« Votre modèle local est moins bon que GPT-4. » C'est exact, et je l'écris. C'est le prix de la conformité RGPD par construction plutôt que par filtrage. La couche `ia_service` isole l'appel : basculer vers un fournisseur distant ne toucherait pas les routes, si LexiCorp acceptait un jour le transfert. (§ 5.2)

« Montrez-nous la fonctionnalité d'enregistrement automatique sur la capture. » Elle n'y est pas, et c'est normal : la capture C-07 montre un document vide, aucune frappe n'a eu lieu. Une capture atteste d'un état, pas d'un comportement — vérifier une fonctionnalité demande de lire le code ou d'exécuter un scénario. C'est la raison d'être du jeu d'essai. (§ 4, § 10)

Si vous ne savez pas répondre « Je ne l'ai pas rencontré sur ce projet. Si je devais le traiter, je commencerais par… » puis donne une démarche, pas une réponse inventée. Et si tu as la réponse dans le dossier : « C'est en § X, je peux vous y renvoyer. »

---

8. Entretien final (20 min)

Pas de question technique ici : le jury vérifie la cohérence du parcours.

- Pourquoi le développement, pourquoi le titre CDA ?
- Qu'est-ce qui vous a le plus marquée dans ce projet ?
- Qu'auriez-vous fait différemment en reprenant au début ?
- Dans quel type de structure vous projetez-vous — et sur quel poste ?
- Comment continuerez-vous à vous former après le titre ?

Pour la troisième, une réponse honnête et préparée : *« J'écrirais le test TU-04 avant la dérive, pas après. J'ai découvert en fin de projet que les textes d'interface et les prompts font partie des spécifications — ils dérivent d'autant plus facilement qu'aucun test ne les couvre. »*